Laboratório: MariaDB e o Custo Real da Representação de Dados

Material de Autoaprendizado — versão estendida

Arquitetura de Computadores para Engenharia
Prof. Robson Lopes

Objetivos

- Instalar e configurar o MariaDB Server no GNU/Linux Debian 12.
- Criar duas versões de uma mesma base de dados: uma com todos os campos em VARCHAR e outra com tipos de dados adequados.
- Inserir os mesmos registros em ambas as tabelas, incluindo campos numéricos fracionários (altura e peso).
- Comparar, de forma mensurável, o espaço em disco ocupado por cada abordagem — tanto no nível lógico (estatísticas do MariaDB) quanto no nível físico (tamanho real do arquivo).
- Relacionar a escolha do tipo de dado com os conceitos de representação binária, número de bits, faixa de valores e representação IEEE-754 vistos em sala.
- Investigar, com ferramentas de inspeção binária (`strings`, `hexdump`), como cada tipo de dado é fisicamente codificado no arquivo do banco — e entender os limites dessa investigação.

1 Instalação do MariaDB

No Debian 12, instale o MariaDB Server usando o gerenciador de pacotes:

```
sudo apt update
sudo apt install mariadb-server -y
sudo mysql_secure_installation
```

Verifique se o serviço está ativo:

```
sudo systemctl status mariadb
```

Acesse o console do MariaDB:

```
sudo mysql -u root -p
```

2 DBA Preguiçoso: Criando a Base Só com VARCHAR

2.1 Criar a base de dados `exemplo01`

```
CREATE DATABASE exemplo01;
USE exemplo01;
```

2.2 Criar a tabela `pessoas_varchar`

Crie a tabela com todos os campos em VARCHAR — já incluindo os campos fracionários `altura` e `peso`, que serão usados mais adiante para investigar representação de ponto flutuante:

```
CREATE TABLE pessoas_varchar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(255),
  email VARCHAR(255),
  cpf VARCHAR(255),
  idade VARCHAR(255),
  altura VARCHAR(255),
  peso VARCHAR(255)
);
```

Observação esperada: nesta tabela, todo campo textual ou numérico foi tratado como cadeia de caracteres de comprimento variável, independentemente do que ele realmente representa. É a abordagem do "DBA preguiçoso": funciona, mas ignora completamente a natureza de cada dado.

3 DBA Experiente: Criando a Base com Tipos Adequados

3.1 Continuar na mesma base de dados

Ao contrário do que se poderia imaginar, o "DBA experiente" não cria um banco separado — ele resolve o mesmo problema, com os mesmos dados, na mesma base `exemplo01`. A diferença entre as duas abordagens está inteiramente na escolha dos tipos de cada campo, não na organização dos bancos:

```
USE exemplo01;
```

3.2 Criar a tabela `pessoas_tipada`

Crie a tabela agora com tipos apropriados a cada dado:

```
CREATE TABLE pessoas_tipada (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100),
  email VARCHAR(100),
  cpf CHAR(11),
  idade TINYINT UNSIGNED,
  altura FLOAT,
  peso FLOAT
);
```

Pontos de discussão antes de prosseguir:

- Por que `cpf` não é `INT`? (zeros à esquerda seriam perdidos)
- Por que `CHAR(11)` e não `VARCHAR(11)` para o CPF? (tamanho fixo, sem overhead de comprimento variável)
- Por que `idade` cabe em `TINYINT UNSIGNED` (1 byte, 0–255)?

3.3 Por que `FLOAT` e não `DOUBLE` ou `DECIMAL`?

- `FLOAT` usa 4 bytes fixos em representação **IEEE-754 de precisão simples** — exatamente o padrão que este laboratório quer expor. Precisão de aproximadamente 7 dígitos decimais significativos, suficiente para valores humanos de altura e peso.

- `DOUBLE` também é IEEE-754, mas de precisão dupla (8 bytes) — overkill para este domínio de dados, só usado se o objetivo for comparar tamanhos entre `FLOAT` e `DOUBLE`.
- `DECIMAL(m,d)` **não** usa IEEE-754: o MariaDB armazena `DECIMAL` em um formato binário próprio de dígitos decimais empacotados. Se o objetivo é localizar o padrão IEEE-754 no arquivo, `DECIMAL` não serve para este experimento (fica reservado para uma discussão futura sobre por que sistemas financeiros evitam `FLOAT`/`DOUBLE`).

Tipo	Bytes	Codificação
FLOAT	4	IEEE-754 precisão simples
DOUBLE	8	IEEE-754 precisão dupla
DECIMAL(m,d)	variável	decimal empacotado (não IEEE-754)

4 Inserção dos Registros

Insira os mesmos registros em ambas as tabelas, agora incluindo altura e peso:

```
INSERT INTO pessoas_varchar (nome, email, cpf, idade, altura, peso) VALUES
('Ana Souza', 'ana.souza@email.com', '12345678901', '28', '1.65', '58.4'),
('Bruno Lima', 'bruno.lima@email.com', '23456789012', '35', '1.78', '82.1'),
('Carla Dias', 'carla.dias@email.com', '34567890123', '41', '1.60', '65.0'),
('Diego Alves', 'diego.alves@email.com', '45678901234', '22', '1.82', '75.3'),
('Elaine Melo', 'elaine.melo@email.com', '56789012345', '30', '1.70', '60.7');

INSERT INTO pessoas_tipada (nome, email, cpf, idade, altura, peso) VALUES
('Ana Souza', 'ana.souza@email.com', '12345678901', 28, 1.65, 58.4),
('Bruno Lima', 'bruno.lima@email.com', '23456789012', 35, 1.78, 82.1),
('Carla Dias', 'carla.dias@email.com', '34567890123', 41, 1.60, 65.0),
('Diego Alves', 'diego.alves@email.com', '45678901234', 22, 1.82, 75.3),
('Elaine Melo', 'elaine.melo@email.com', '56789012345', 30, 1.70, 60.7);
```

Confira os dados inseridos:

```
SELECT * FROM pessoas_varchar;
SELECT * FROM pessoas_tipada;
```

5 Gerando Volume de Dados para uma Comparação Realista

Por que isso é necessário: com apenas 5 registros, a diferença real de bytes entre os dois esquemas é da ordem de dezenas de bytes — completamente mascarada pelo overhead fixo de metadados que o InnoDB aloca para qualquer tablespace novo (páginas de cabeçalho, bitmap do insert buffer, página INODE, raiz do índice), da ordem de dezenas de KB. Isso significa que `ls -lh` nos arquivos `.ibd` provavelmente mostrará tamanhos iguais ou quase iguais entre as duas tabelas com poucos registros — não porque o experimento esteja errado, mas porque a granularidade de alocação em páginas/extents esconde diferenças pequenas.

Para que a diferença apareça também no tamanho físico do arquivo, gere alguns milhares de registros com o script auxiliar (Python + Faker) fornecido junto a este material.

5.1 Preparando o usuário de banco para o script

O script se conecta ao MariaDB via TCP/IP (biblioteca PyMySQL), e **não** via socket Unix. Isso importa porque, por padrão no Debian, o usuário `root@localhost` do MariaDB usa o plugin de autenticação `unix_socket` — que ignora qualquer senha e só aceita login quando o processo cliente roda com o mesmo usuário do sistema operacional, via socket Unix (`sudo mysql -u`

`root`). Uma conexão TCP com usuário e senha para esse usuário sempre falha com o erro `1698 (Access denied)`, independentemente da senha estar certa — não é um problema de rede nem de configuração em `my.cnf`.

A solução é conceder ao usuário do sistema operacional que vai rodar o script (aqui, `sysadmin`) uma conta do MariaDB que aceite autenticação por senha via TCP:

```
sudo mysql -u root

CREATE USER 'sysadmin'@'127.0.0.1' IDENTIFIED BY 'SUASENHA';
GRANT ALL PRIVILEGES ON exemplo01.* TO 'sysadmin'@'127.0.0.1';
FLUSH PRIVILEGES;
```

Atenção a um detalhe que confunde muita gente: no `CREATE USER`, o host `localhost` tem um significado especial no MySQL/MariaDB — ele corresponde *apenas* a conexões via socket Unix. Uma conexão TCP para `127.0.0.1`, mesmo sendo a própria máquina, é tratada como uma conexão de rede separada. Por isso o usuário precisa ser criado explicitamente como `'sysadmin'@'127.0.0.1'` (ou `'sysadmin'@'%'` para aceitar de qualquer host) — `'sysadmin'@'localhost'` **não** funcionaria para o script.

5.2 Executando o script

```
pip install pymysql Faker --break-system-packages
python3 gerar_massa.py --user sysadmin --password 'SUASENHA' --database exemplo01 --
    registros 10000
```

O script insere os mesmos valores gerados em ambas as tabelas (convertendo para string na versão VARCHAR), em lotes, e ao final imprime os comandos de verificação necessários.

6 Comparando o Tamanho em Disco

6.1 Opção A — via SQL (recomendada, funciona em qualquer instalação)

Antes de comparar tamanhos, confirme que as duas tabelas têm o mesmo número de linhas — é uma checagem de sanidade rápida: se os volumes forem diferentes, qualquer comparação de espaço fica invalidada, já que uma tabela naturalmente ocuparia mais espaço só por ter mais registros, não por causa do tipo de dado:

```
SELECT COUNT(*) FROM pessoas_varchar;
SELECT COUNT(*) FROM pessoas_tipada;
```

Com os volumes confirmados, force a atualização das estatísticas do InnoDB (que ficam em cache e podem estar desatualizadas):

```
USE exemplo01;
ANALYZE TABLE pessoas_varchar, pessoas_tipada;

SELECT table_name,
       data_length AS bytes_dados,
       index_length AS bytes_indice,
       (data_length + index_length) AS total_bytes
FROM information_schema.tables
WHERE table_schema = 'exemplo01';
```

Atenção a onde você executa cada comando: o `ANALYZE TABLE` precisa ser executado estando no banco onde as tabelas realmente existem (`exemplo01`) — **não** dentro de `information_schema`. Se você estiver com `USE information_schema;` ativo e rodar o comando sem qualificar o schema, o MariaDB vai procurar `pessoas_varchar`/`pessoas_tipada` dentro de

`information_schema` e não vai encontrá-las, já que elas não existem lá. O `information_schema` é um banco virtual, somente leitura, que expõe metadados de *todos* os bancos do servidor — é por isso que o `SELECT` seguinte, ao referenciar `information_schema.tables` com o nome do schema qualificado, funciona não importa em qual banco você esteja no momento.

Nota: para o motor InnoDB, `data_length`/`index_length` são *estimativas* calculadas a partir do comprimento médio de linha, não uma leitura exata e literal do arquivo (a documentação oficial do MariaDB é explícita quanto a essa aproximação). Ainda assim, refletem o tamanho lógico da linha — exatamente onde a diferença de tipo de dado aparece, independentemente da granularidade de alocação física.

6.2 Opção B — direto no arquivo (InnoDB, `innodb_file_per_table`)

```
sudo ls -lh /var/lib/mysql/exemplo01/
```

Resposta esperada: os arquivos `pessoas_varchar.ibd` e `pessoas_tipada.ibd` devem aparecer listados, cada um com seu tamanho em bytes/KB. Com volume suficiente de registros (Seção 5), a diferença deve ser visível; com poucos registros, pode não ser — e essa observação, por si só, é um resultado válido a ser discutido (ver Seção 8).

Anote os dois valores exatos — eles serão necessários na atividade do AVA.

6.3 Estudo de escala: como a diferença se comporta em volumes maiores

Um resultado importante que aparece ao testar volumes crescentes: a diferença entre as duas tabelas não cresce de forma visível e contínua no arquivo físico. Ela fica escondida atrás de **camadas sucessivas de granularidade de alocação** do InnoDB:

- Com poucas centenas ou milhares de registros, ambas as tabelas cabem na alocação inicial mínima e aparecem com o mesmo tamanho físico.
- Em um patamar intermediário, as duas podem convergir para o mesmo número de **extents** (unidades de alocação de 1 MiB no InnoDB, agrupando 64 páginas de 16 KB) — mesmo usando quantidades diferentes de espaço *dentro* do último extent.
- Só quando a diferença acumulada ultrapassa o tamanho de um extent inteiro é que o arquivo físico de uma tabela passa a ocupar uma unidade de alocação a mais que a outra — e a diferença aparece de forma discreta (em saltos de 1 MiB), não proporcional.

Para observar essa progressão de forma controlada, rode o experimento em múltiplos volumes com o script `estudo_escala.py`, fornecido junto a este material. Ele trunca as tabelas, insere um volume novo, mede as estatísticas lógicas (`information_schema`) e o tamanho físico real do `.ibd` a cada rodada:

```
sudo python3 estudo_escala.py --user sysadmin --password 'SUASENHA'
```

Atenção: o script executa `TRUNCATE TABLE` antes de cada rodada — ele apaga os dados anteriores das duas tabelas a cada novo volume testado. Rode-o com `sudo`, já que a medição física precisa ler arquivos que pertencem ao usuário `mysql`.

Preencha a tabela abaixo com os resultados (ou use diretamente o CSV gerado pelo script):

Registros	data_length VARCHAR (B)	data_length TIPADA (B)	.ibd físico VARCHAR (B)	.ibd físico TIPADA (B)	Diferença física (B)
10.000					
100.000					
200.000					
500.000					
1.000.000					

Para uma medida que não sofre esse efeito de arredondamento, calcule também o tamanho lógico do conteúdo bruto (sem overhead do InnoDB), que cresce de forma linear e previsível com o volume:

```
SELECT SUM(LENGTH(nome)+LENGTH(email)+LENGTH(cpf)+LENGTH(idade)+LENGTH(altura)+LENGTH(
    peso))
      AS bytes_conteudo
FROM pessoas_varchar;

SELECT SUM(LENGTH(nome)+LENGTH(email)+LENGTH(cpf)+LENGTH(idade)+LENGTH(altura)+LENGTH(
    peso))
      AS bytes_conteudo
FROM pessoas_tipada;
```

6.4 Automatizando a verificação

Os comandos das Seções 6.1, 6.2 e 7.2–7.3 podem ser rodados de uma vez com o script `verificar.sh`, fornecido junto a este material. Ele apenas consulta e mede o que já existe — não trunca nem insere nada — e imprime a saída já numerada em blocos `[1]`–`[5]`, prontos para copiar direto na atividade do AVA:

```
./verificar.sh -u sysadmin -p 'SUASENHA' -d exemplo01
```

Para incluir também a busca do padrão IEEE-754 (bloco `[5]`), passe um valor de altura ou peso com `-f`:

```
./verificar.sh -u sysadmin -p 'SUASENHA' -f 1.65
```

7 Investigando a Codificação Física dos Dados

Esta seção conecta a teoria de representação binária vista em sala com uma inspeção direta dos bytes gravados em disco pelo MariaDB.

7.1 Por que o GDB não é a ferramenta certa aqui

O GDB foi projetado para inspecionar a memória de um *processo em execução* (ou um core dump), não para abrir um arquivo estático em disco e interpretar seus bytes diretamente. Para essa tarefa, ferramentas de inspeção de arquivo são mais diretas e apropriadas:

- `strings -a arquivo.ibd` — localiza sequências de bytes imprimíveis (texto).
- `hexdump -C arquivo.ibd` ou `xxd arquivo.ibd` — exibe o conteúdo bruto em hexadecimal.

O GDB continua sendo a ferramenta certa para a demonstração complementar de IEEE-754 em memória de processo, descrita na Seção 7.4.

7.2 Por que `strings` encontra texto mas não números

O comando `strings` procura sequências de bytes que correspondam a caracteres ASCII/UTF-8 imprimíveis (tipicamente 4 ou mais bytes consecutivos nessa faixa). Um campo `VARCHAR`/`CHAR` é gravado como texto puro — por isso `strings` encontra facilmente nomes e e-mails. Já um `TINYINT`, `INT` ou `FLOAT` é gravado como a representação binária pura do valor numérico: o byte correspondente normalmente **não** cai na faixa de caracteres imprimíveis e, portanto, **não aparece** na saída do `strings`.

Exercício de observação: execute `strings -a` no arquivo `pessoas_tipada.ibd` e compare o que aparece (nomes, e-mails) com o que não aparece (idade, altura, peso). Essa ausência é, por si só, uma evidência experimental da diferença entre "dado como sequência de caracteres" e "dado como valor binário".

7.3 Localizando o padrão IEEE-754 com `hexdump`

Para localizar um valor fracionário específico no arquivo, calcule antes o padrão de bytes esperado:

```
import struct
struct.pack('>f', 1.65).hex() # IEEE-754 simples, big-endian
```

E procure esse padrão no dump:

```
hexdump -C pessoas_tipada.ibd | grep -i "<padrao_de_bytes>"
```

Dois cuidados importantes ao interpretar o resultado:

1. **Ordem de bytes (endianness):** o InnoDB grava campos numéricos de largura fixa em *big-endian*, independentemente da arquitetura do processador que está rodando o servidor (tipicamente little-endian em x86-64) — isso é proposital, para permitir comparação byte-a-byte correta nas B-trees dos índices. Ou seja, o valor **não** aparece no arquivo na mesma ordem de bytes que a CPU usa nativamente na memória de um processo comum.

2. **Estrutura de página:** o arquivo `.ibd` não é um dump direto tipo struct-em-C. É organizado em páginas de 16 KB, com cabeçalhos de página, diretório de registros e formato de linha COMPACT/DYNAMIC (incluindo colunas ocultas do InnoDB, como `trx_id` e `roll pointer`). Isso explica por que localizar um campo específico é mais trabalhoso do que em um arquivo binário simples.

Caso mais limpo para observação: o campo `idade` (`TINYINT UNSIGNED`) é o mais simples de todos — por não ter sinal, é gravado como o valor binário puro em 1 byte, sem nenhuma transformação adicional. Um valor como 28 deve aparecer literalmente como o byte `0x1C` no arquivo. *(Nota: para tipos **com** sinal, alguns motores de armazenamento aplicam uma inversão do bit de sinal para fins de ordenação em índice; essa nuance está fora do escopo garantido deste laboratório e pode variar por versão — trate como investigação aberta, não como fato assumido.)*

7.4 Demonstração complementar: IEEE-754 com GDB

Para uma demonstração de IEEE-754 sem a complexidade do formato de armazenamento do InnoDB, use um programa C mínimo:

```
// altura.c
int main() {
    float altura = 1.65f;
    return 0;
}
```

```
gcc -g -o altura altura.c
gdb ./altura
(gdb) break main
(gdb) run
(gdb) next
(gdb) p/x altura
(gdb) x/4xb &altura
```

Compare o padrão de bytes obtido aqui (memória do processo, ordem nativa da CPU) com o padrão calculado via `struct.pack` em Python — e, se desejar, com o que for encontrado no arquivo `.ibd` (Seção 7.3). A comparação entre os três cenários é o fechamento natural da discussão de endianness pendente da unidade.

8 Exercícios Propostos

1. Execute a consulta da Opção A e registre, em bytes, os valores de `data_length` e `index_length` de cada tabela.
2. Execute o comando da Opção B e registre o tamanho de cada arquivo `.ibd`.
3. Calcule a diferença, em bytes, entre `pessoas_varchar` e `pessoas_tipada`, tanto pela Opção A quanto pela Opção B. As duas diferenças são iguais? Se não, por quê?
4. Projete: se a tabela tivesse 1.000.000 de registros, qual seria o impacto aproximado dessa diferença?
5. Explique, em suas palavras, por que `TINYINT UNSIGNED` (1 byte) e `VARCHAR(255)` para o mesmo dado (idade) não ocupam o mesmo espaço, ainda que o valor armazenado seja idêntico (por exemplo, 28).
6. Execute `strings -a` no arquivo `pessoas_tipada.ibd`. Quais campos aparecem na saída e quais não aparecem? Explique o motivo com base na natureza binária dos tipos numéricos.
7. Repita a inserção com apenas 5 registros e depois com o volume gerado pelo script (Seção 5). A diferença de tamanho *físico* do arquivo (Opção B) muda de forma proporcional? Relacione sua resposta com o conceito de granularidade de alocação (páginas/extents).
8. (Desafio) Usando `hexdump` e o valor calculado com `struct.pack('>f', ...)`, tente localizar o padrão de bytes de um valor de `altura` ou `peso` no arquivo `.ibd`. Documente se encontrou o padrão exato, um padrão parcial, ou nenhum — e formule uma hipótese para o resultado observado.
9. Rode o `estudo_escala.py` e preencha a tabela da Seção 6.3. Em que ponto a diferença física entre as tabelas passa a aparecer? Ela cresce de forma proporcional ao volume, ou em saltos? Relacione com o conceito de extents (unidades de alocação de 1 MiB no InnoDB).

Checklist de Observações

- O tipo de dado define quantos bits/bytes são reservados por campo — o mesmo raciocínio de dimensionamento (n bits $\rightarrow 2^n$ combinações) usado na representação binária de inteiros.

- `VARCHAR` armazena, além do conteúdo, informação sobre o comprimento da string — overhead que `CHAR` de tamanho fixo não possui quando o dado tem tamanho constante (como o CPF).
- `FLOAT` usa representação IEEE-754 de precisão simples (4 bytes); `DECIMAL` usa uma codificação binária de dígitos decimais, não IEEE-754 — são conceitos diferentes, apesar de ambos representarem números fracionários.
- Ferramentas de inspeção de arquivo (`strings`, `hexdump`) tornam visível, de forma direta, a diferença entre dado armazenado como texto e dado armazenado como valor binário — mas a estrutura interna de páginas do InnoDB e a ordem de bytes (big-endian) impõem limites reais a essa investigação.
- Diferenças pequenas no tamanho lógico de uma linha podem não se refletir imediatamente no tamanho físico do arquivo, por causa da granularidade de alocação em páginas/extents — um fenômeno análogo à fragmentação interna vista em outros contextos de arquitetura de computadores.
- A escolha correta do tipo de dado é uma decisão de engenharia com impacto real e mensurável em armazenamento e desempenho.

Conclusão

Com este experimento, o estudante comprova, de forma prática e mensurável, que a escolha do tipo de dado em um banco de dados não é uma formalidade sintática: ela reflete diretamente conceitos de representação binária, quantidade de bits necessários e alocação de memória — os mesmos princípios que regem como uma variável `int` ou `float` é armazenada em um programa em linguagem C. A extensão com campos fracionários e inspeção binária direta aprofunda essa conexão, expondo os alunos à representação IEEE-754 real em um sistema de produção e aos limites práticos de se investigar formatos de armazenamento complexos — uma habilidade de engenharia tão importante quanto a teoria em si.